Ver en el repositorio: CLASE3/00_ejemplos_base y CLASE3/01_ejemplos_ampliados

Una lista reúne varios valores que necesitamos recorrer

LISTA

```
costos: list[float] = [
    45000.0,
    19200.0,
    8500.0
]

print(costos[0])
print(len(costos))
```

Índice 0

primer elemento

len()

cantidad de elementos

La lista puede crecer:
costos.append(12000.0)

Ver en el repositorio: CLASE3/00_ejemplos_base y CLASE3/01_ejemplos_ampliados

Una tupla reúne datos relacionados en una posición fija

TUPLE

```
rubro: tuple[str, float] = (
    "Espacio",
    120000.0,
)

print(rubro[0])
print(rubro[1])
```

Dos datos que forman una unidad

En este caso, cada rubro guarda:

0 nombre del rubro
1 costo del rubro

Una lista puede contener varias tuplas:

rubros: list[tuple[str, float]]

Ver en el repositorio: CLASE3/00_ejemplos_base y CLASE3/01_ejemplos_ampliados

for repite una acción para cada elemento

RECORRIDO Y ACUMULACIÓN

```
costos: list[float] = [45000.0, 19200.0, 8500.0]

total: float = 0.0

for costo in costos:
    total += costo
    print(f"Procesado: ${costo:,.2f}")

print(f"Total: ${total:,.2f}")
```

costo toma un valor distinto en cada vuelta.

total existe antes de comenzar el recorrido.

El bloque indentado se repite.

Antes de ejecutar: ¿qué valor tendrá total después de la segunda vuelta?

Ver en el repositorio: CLASE3/00_ejemplos_base y CLASE3/01_ejemplos_ampliados

El for puede separar los valores de cada tupla

DESEMPAQUETADO

```
for nombre, costo in rubros:
    print(nombre)
    print(costo)

for _, costo in rubros:
    total += costo
```

nombre, costo

Usamos los dos valores de la tupla.

_, costo

El guion bajo indica que el primer valor existe, pero no necesitamos utilizarlo.

Ver en el repositorio: CLASE3/00_ejemplos_base y CLASE3/01_ejemplos_ampliados

while repite mientras una condición sea True

REPETICIÓN CON CONDICIÓN

```
pendientes: int = 3

while pendientes > 0:
    print(f"Quedan {pendientes}")
    pendientes -= 1

print("Proceso terminado")
```

Usar while cuando no sabemos de antemano cuántas repeticiones habrá.

La condición debe poder cambiar.
Si no, el bucle puede no terminar.

Ver en el repositorio: CLASE3/00_ejemplos_base y CLASE3/01_ejemplos_ampliados

Una función le da nombre a una operación reutilizable

PARÁMETRO · RETORNO

```
costos: list[float] = [
    45000.0,
    19200.0,
    8500.0,
]

def calcular_total(costos: list[float]) -> float:
    total: float = 0.0
    for costo in costos:
        total += costo
    return total

total_produccion: float = calcular_total(costos)
print(f"Total de la producción: ${total_produccion:,.2f}")
```

- Recibe datos mediante parámetros.
- Encapsula una tarea concreta.
- Devuelve un resultado con return.
- Puede probarse de manera independiente.

Ver en el repositorio: CLASE3/00_ejemplos_base y CLASE3/01_ejemplos_ampliados

Separar funciones permite cambiar una regla sin reescribir todo

DOS RESPONSABILIDADES

```
def aplicar_descuento(subtotal: float, porcentaje: float) -> float:
    descuento: float = subtotal * porcentaje / 100
    return subtotal - descuento

def entra_en_presupuesto(total: float, presupuesto: float) -> bool:
    return total <= presupuesto

subtotal: float = 75000.0
porcentaje_descuento: float = 10.0
presupuesto: float = 68000.0

total_final: float = aplicar_descuento(subtotal, porcentaje_descuento)
presupuesto_ok: bool = entra_en_presupuesto(total_final, presupuesto)

print(f"Subtotal: ${subtotal:,.2f}")
print(f"Total final: ${total_final:,.2f}")
print(f"¿Entra en el presupuesto?: {presupuesto_ok}")
```

Expansión: escribí una tercera función que produzca el mensaje final.

Cada función debería responder una pregunta clara.

Ver en el repositorio: CLASE3/00_ejemplos_base y CLASE3/01_ejemplos_ampliados

Clases: datos y métodos

CREAMOS LA CLASE

```
# “class” indica que vamos a crear una clase.
class Producto:
    # __init__ se ejecuta al crear un producto.
    def __init__(
        self,
        nombre: str,
        precio_unitario: float,
        cantidad: int,
    ) -> None:
        # Guarda los datos dentro del objeto.
        self.nombre: str = nombre
        self.precio_unitario: float = precio_unitario
        self.cantidad: int = cantidad

    # Define una acción del producto.
    def calcular_subtotal(self) -> float:
        return self.precio_unitario * self.cantidad
```

INSTANCIO LA CLASE EN UN OBJETO Y LA USO

```
lampara: Producto = Producto(
    nombre="Lámpara",
    precio_unitario=25000.0,
    cantidad=3,
)

print(f"Producto: {lampara.nombre}")
print(f"Precio unitario: ${lampara.precio_unitario:,.2f}")
print(f"Cantidad: {lampara.cantidad}")

subtotal: float = lampara.calcular_subtotal()
print(f"Subtotal: ${subtotal:,.2f}")
```

Van a revisar tres programas completos, los cuales se encuentran en dificultad ascendente

01

Presupuesto de actividad

variables · listas · funciones · if

02

Planificador de producción

clase · materiales · costo · viabilidad

03

Pedidos y stock

dos clases · for · while · búsqueda

Los tres programas pueden encontrarlos en el repositorio en la CLASE3/ejercicio

Cada programa permite trabajar con un nivel diferente

01	**Presupuesto de actividad**	Base común	Modificar rubros, costos e imprevistos
02	**Planificador de producción**	Intermedio	Calcular precio de venta y margen
03	**Pedidos y stock**	Avanzado	Agregar descuentos o reponer stock

Las variantes resueltas se consultan después de intentar y comprobar una solución propia.

Vamos a trabajar con el siguiente criterio: primero comprendemos el programa; después lo modificamos

01

EJECUTAR

Correr todas las celdas y observar el resultado.

02

LOCALIZAR

Encontrar listas, loops, funciones, clases y condiciones.

03

MODIFICAR

Cambiar datos y resolver una propuesta de expansión.

Quizás se encuentren con algunos errores, esto es una lista básica de guía

NameError

se usó un nombre que no existe todavía, deberían crear la variable

ValueError

la conversión no pudo realizarse

TypeError

la operación no tiene sentido para esos tipos

IndentationError

el bloque no está correctamente indentado. Un clásico problema de identación, recuerden que python es sensible a la identación

Les dejo la página de python con los errores y excepciones con los que se pueden encontrar:

https://docs.python.org/es/3/tutorial/errors.html

Una vez revisado, ejecuta y cambiado el código pueden ayudarse con IA para trabajar con los archivos y explicar, revisar, modificar y comprobar nuevas versiones.

1	**EXPLICAR**	¿Qué hace cada bloque?
2	**REVISAR**	¿Qué error o caso falta contemplar?
3	**MODIFICAR**	¿Qué líneas debemos cambiar?
4	**COMPROBAR**	¿Con qué casos verificamos el resultado?

REGLA DE TRABAJO

La IA no reemplaza todo el programa.

Le pedimos una explicación o una modificación acotada y luego probamos el resultado.

En el repositorio, dentro de la carpeta CLASE3/ejercicios, van a poder encontrar una notebook con información sobre como trabajar con IA para el desarrollo de código. En este caso lo vamos a usar como Copiloto. La notebook se denomina 04_trabajo_con_IA_sobre_programas

Pueden ir armando un primer informe sobre el trabajo con estos ejemplos: el resultado incluye el código y las decisiones tomadas

01 Programa inicial

02 Cambio intentado sin IA

03 Prompt utilizado

04 Respuesta recibida

05 Código finalmente incorporado

06 Casos de prueba

Explicación final: qué hice, qué aportó la IA y cómo comprobé que funcionaba.